Object-Oriented Programming Cont..

1. Quick recap: classes and objects

A class is a blueprint; an object is an instance created from that blueprint.

For example, `Student` is a class, while `s1` and `s2` are individual student objects.

```
class Student {
public:
    string name;
    int marks;

    void display() {
        cout << name << " " << marks << endl;
    }
};

int main() {
    Student s1;
    s1.name = "Riya";
    s1.marks = 90;

    s1.display();
}
```

2. The four pillars of OOP

The four pillars are:

1. Encapsulation: Bundle data and methods; control access.

Example: bank account balance.

2. Abstraction: Expose essential operations; hide implementation details.

Example: ATM.

3. Inheritance: Create a specialized class from an existing class.

Example: Dog inherits from Animal.

4. Polymorphism : One interface, different behaviours.

Example: different implementations of `speak()`.

2.1 Encapsulation

Definition: Encapsulation combines data and the functions that operate on that data inside a class, while controlling access to the data.

Real-life example: bank account

Imagine a bank account with ₹5,000.

If the balance were publicly accessible, any code could change it to −₹10,000. A better design allows users to deposit and withdraw through controlled functions.

```
class BankAccount {
private:
    double balance = 0;

public:
    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    bool withdraw(double amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }

        balance -= amount;
        return true;
    }

    double getBalance() {
        return balance;
    }
};
```

Explanation:

- `private`: outside code cannot directly access `balance`.

- `public`: outside code can call `deposit()`, `withdraw()` and `getBalance()`.
- Validation prevents invalid transactions.
- The class keeps its data and related behaviour together.

Important distinction

Encapsulation is not just making every variable private. It is about bundling state and behaviour and controlling how that state is accessed or modified.

2.2 Abstraction

Definition: Abstraction means showing only the essential features to the user while hiding the unnecessary implementation details.

Real-life example: ATM

When you withdraw money, you:

1. Insert your card.
2. Enter your PIN.
3. Select an amount.
4. Receive cash.

You do not need to know how the ATM communicates with the bank server or verifies the transaction internally.

```
class ATM {
public:
    void withdrawMoney() {
        verifyUser();
        checkBalance();
        dispenseCash();
    }

private:
    void verifyUser() {
        cout << "Verifying user\n";
    }

    void checkBalance() {
        cout << "Checking balance\n";
    }

    void dispenseCash() {
```

```
        cout << "Dispensing cash\n";
    }
};
```

The user calls `withdrawMoney()` without needing to understand each internal step.

How is abstraction implemented in C++?

- Classes and access specifiers.
- Public interfaces that hide internal implementation.
- Abstract classes and pure virtual functions.

Encapsulation vs abstraction

Encapsulation	Abstraction
Bundles data and behaviour	Hides implementation complexity
Controls access to internal state	Exposes essential operations
Example: private bank balance	Example: ATM withdrawal interface

Easy memory trick: Encapsulation = protect and bundle. Abstraction = simplify and hide.

2.3 Inheritance

Definition: Inheritance allows a new class, called the derived class, to reuse and extend accessible members of an existing base class.

Real-life example

A Dog is an Animal. A dog can perform general animal behaviours, such as eating, and can also have its own behaviours, such as barking.

```
class Animal {
public:
    void eat() {
```

```
        cout << "Eating\n";
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << "Woof!\n";
    }
};

int main() {
    Dog d;
    d.eat();   // Inherited from Animal
    d.bark();  // Defined in Dog
}
```

Types of inheritance

1. Single inheritance

One base class and one derived class.

A → B

2. Multilevel inheritance

A class inherits from a derived class.

A → B → C

3. Multiple inheritance

One class inherits from multiple base classes.

A + B → C

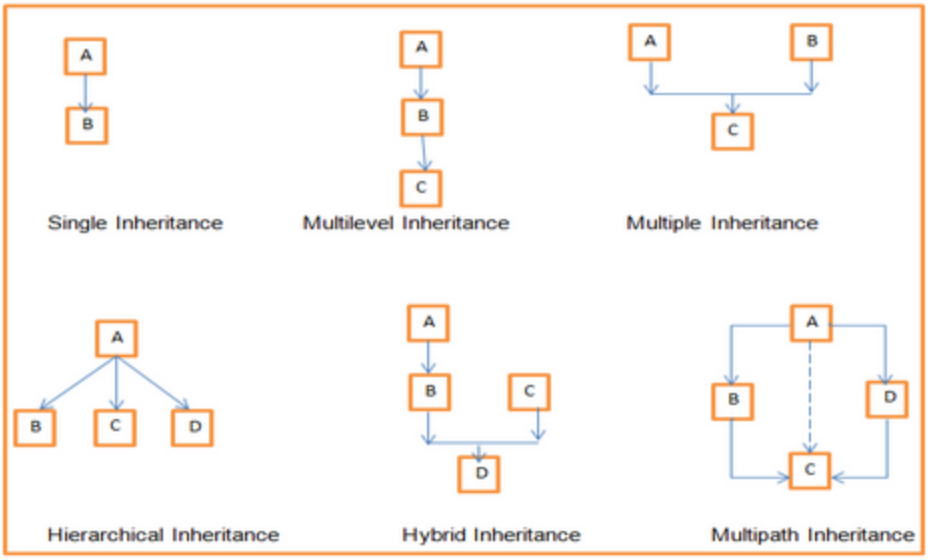
4. Hierarchical inheritance

Multiple classes inherit from one base class.

A → B and A → C

5. Hybrid inheritance

A combination of two or more inheritance types.



V.Imp: https://www.geeksforgeeks.org/cpp/diamond-problem-in-cpp/

2.4 Polymorphism

Definition: Polymorphism means “many forms.” The same interface can represent different behaviours.

There are two major forms.

A. Compile-time polymorphism

The compiler selects the relevant operation based on the code and arguments.

Common examples:

- Function overloading.
- Operator overloading.

Function overloading

```
int add(int a, int b) {
    return a + b;
}

double add(double a, double b) {
    return a + b;
}
```

```
int add(int a, int b, int c) {
    return a + b + c;
}
```

All three functions are called `add`, but their parameter lists differ.

Important: You cannot overload functions by changing only their return types.

B. Run-time polymorphism

The implementation is selected at run time through virtual functions and overriding.

```
class Animal {
public:
    virtual void speak() {
        cout << "Animal sound\n";
    }

    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void speak() override {
        cout << "Woof!\n";
    }
};

int main() {
    Dog d;
    Animal* p = &d;
`
    p->speak();  // Woof!
}
```

Explanation:

1. `d` is a Dog object.
2. `p` is a pointer of type `Animal*`.
3. The pointer refers to the Dog object.
4. Since `speak()` is virtual, the Dog implementation is called.

If `speak()` were not virtual, the call through `Animal*` would use the base-class implementation.

The `override` keyword helps the compiler detect mistakes in the derived method's signature.

Overloading vs overriding

Overloading	Overriding
Same function name, different parameter lists	Derived class redefines a virtual base-class function
Usually compile-time polymorphism	Run-time polymorphism through a base pointer/reference
Can occur within one class	Requires inheritance
Return type alone cannot distinguish overloads	Matching virtual function signature is important